UNIFIED FORMAL AUDIT SPECIFICATION — IMPLEMENTATION CORRESPONDENCE (UFAS-IC v1.0)

Specification Author:
Vladimir Khomyakov / Independent Researcher
ORCID iD: https://orcid.org/0009-0006-3074-9145

Module: UFAS-IC — Implementation Correspondence
Companion module: UFAS-LC — Logical Closure
The modules are complementary: each is self-contained and executed on its own, and neither supersedes the other. Version numbers are assigned per module.

TASK

Verify the correspondence between the VERBAL/MATHEMATICAL description and the ACTUAL STRUCTURE of the accompanying implementation (code, functions, procedures, algorithms, computational blocks, tables, protocols, etc.). Objective: to establish whether the implementation does what the text asserts about the computations or procedures being performed.

Do not check numerical values (cross-checking the numbers in the text against logs and output—"numeric-grep"—is a separate task); instead, check structure: arguments and input data; the order of steps; dependencies between operations; conditions and branching; what is computed or verified, and by what means; and the correspondence of the stated result to the procedure actually performed. This is closer to paragraph-by-paragraph reading, but applied selectively: only where numerical cross-checking cannot, by definition, settle the question.

The task is self-contained: there are no preceding steps.

INPUT

The manuscript consists of the paper's files located in the Project and/or in the chat (attached or pasted), e.g., `.tex` sources and any accompanying scripts or run outputs. Files in the Project count as provided: check both places before reporting anything missing. Copies of this specification (`UFAS_*`) are not part of the manuscript. Use only the manuscript files; if none are found, state where you looked and stop.

STEP 1 — INVENTORY (BEFORE VERIFICATION)

Determine for yourself which passages of the manuscript contain a verifiable description (a computation, lemma, theorem, proof, algorithm, formula, numerical procedure, or a statement of what was verified and how) and which elements of the implementation correspond to them. Do not restrict yourself to literal markers such as `computation` or `lemma`; do not presuppose particular sections, a particular language, or the presence of code.

List:

(a) the selected passages;
(b) the implementations referenced by the text, each marked "present" or "ABSENT from the materials" (including files referenced by include commands but not supplied).

STEP 2 — VERIFICATION

For each passage in the inventory, produce one row of the following table:

No.	Claim in the text (location)	Implementation (file:element:lines)	What the implementation actually does	Verdict	Affects the stated conclusion?

The verdict is exactly one of:

- **MATCH**
- **DISCREPANCY**
- **PARTIAL** (the text omits or misrepresents a step, parameter, or condition)
- **NO IMPLEMENTATION**
- **INCONCLUSIVE**

Rules:

1. Accompany every verdict other than NO IMPLEMENTATION with a quotation of 1–2 lines from the implementation (or a precise reference to the fragment). A verdict without a quotation does not count: assign INCONCLUSIVE.
2. If no structural counterpart exists, do not invent one: mark NO IMPLEMENTATION and verify only those aspects for which there is a factual basis.
3. If the wording of the task is inapplicable to the given type of material, adapt it to the actual structure of the manuscript while preserving the purpose of the verification; briefly explain the adaptation.
4. Do not run the implementation if a conclusive verdict can be reached without running it; if you ran it, state why.
5. Report only; do not correct or rewrite anything.

STEP 3 — SUMMARY

A count of verdicts of each type; a list of DISCREPANCY and PARTIAL verdicts, noting their impact on the conclusions; a list of missing implementations; what remained unverified, and why.

END OF SPECIFICATION